TripMate AI: Intelligent Group Activity Planning Platform

Powered by a Unified Deep-Reasoning Agent

Project Idea, Features & Technical Architecture Proposal for Supervisor

Project Team (6 Members): Ta Quang Huy Ha Dang Huy Pham Dinh Anh Duong Nguyen Minh Duc Nguyen Tung Duong Dinh Tien Luan

1. Introduction & Project Idea

1.1. The Problem

Organizing activities for a group of 3–10+ people (a trip, a dinner, a weekend hangout) is painful in practice:

- **Manual research** across maps, booking sites and reviews takes days.
- **Schedule overlap** across members with different free windows ($\pm\delta$ tolerance) is a combinatorial puzzle.
- **Conflicting preferences** (beach vs. mountain, budget vs. comfort) lead to decision paralysis.
- **Fragmented chat** on Zalo/Messenger buries plans and links.
- **One organizer** carries the entire load and gets blamed if the group is unhappy.
- **Expense splitting** after the trip is tedious and awkward.

1.2. The Solution

TripMate AI turns group planning into a guided, consensus-driven flow in under 15 minutes: members chat freely in a shared room → an AI agent asks clarifying questions → the agent researches and proposes an itinerary → the group votes leg-by-leg → the plan is locked and expenses are auto-settled.

Figure 1: End-to-end collaborative planning workflow

2. Platform Features

- **Group Room:** Shared chat room, invite by link. Members chat freely — budget, dietary needs, dates and preferences are shared naturally in conversation or collected by the agent when it detects they are needed. 6 event types: `TRAVEL`, `DINING`, `HANGOUT`, `ENTERTAINMENT`, `SIGHTSEEING`, `CUSTOM`. Roles: Host, Member, Viewer.
- **AI Planning Hub:** One-click itinerary generation, live flight/hotel/ticket lookup, dietary-aware dining suggestions, visa/passport checks.
- **Voting & Customization:** Leg-by-leg UP/DOWN voting with instant alternatives, live AI chat to tweak the plan, manual drag-and-drop editor.
- **Map & Utilities:** Interactive map (Mapbox), weather-based packing checklist, PDF export, Vietnamese/English UI.
- **Expense Splitting:** Tracks advances and payments, supports equal/exact/percentage splits, computes minimum peer-to-peer settlements.

3. The AI Agent: Tools, Skills & Operational Flow

3.1. Model Layer

The agent is powered by **DeepSeek-V4**, utilizing two operational tiers within a unified context:

- **Flash** — fast chat interaction, room message parsing, and tool-call formatting.
- **Pro** — deep reasoning for constraint solving, spatial planning, and preference arbitration.

3.2. Tools (External Capabilities)

Tools are typed callable functions providing real-time data:

Tool	Function & Parameters → Output	Provider
`search_places`	query, location, radius → POIs, ratings, opening hours	Google Places
`search_flights`	origin, destination, dates, pax → flight schedules & fares	Amadeus
`search_hotels`	location, dates, guests → lodging options & rates	Booking.com
`search_activities`	location, category → tickets, tours & activities	Klook
`check_visa`	nationality, destination → visa & passport requirements	Passport Index
`get_weather`	location, date range → forecast & weather alerts	OpenWeatherMap
`get_route`	origin, destination, mode, departure_time → duration, distance	Mapbox

Execution policy: Independent tool calls (e.g., weather + flights + hotels) run **in parallel**; dependent calls (e.g., routing between chosen POIs) execute sequentially. All calls are logged in `agent_logs`.

3.3. Agent Skills

Internal capabilities governing agent behavior beyond raw API querying:

- **Research skill:** contextual query formulation, multi-source retrieval, and candidate ranking (Section 3.4).
- **Clarification skill:** extracts stated preferences from chat; only queries missing critical parameters without repetitive intake forms.
- **Conflict-mediation skill:** negotiates compromises for conflicting preferences (e.g., beach vs. mountain) through hybrid options.
- **Local-repair skill:** patches downvoted itinerary legs with pre-scored alternatives without full re-planning.
- **Validation skill:** verifies feasibility (prices > 0, dates within window, valid opening hours) before presenting options.

3.4. Research Skill: Finding Places, Hotels & Restaurants

The Research skill transforms group preferences into vetted, ranked candidates through a streamlined process:

1. **Contextual Querying:** Embeds dietary needs, group size, and location proximity into queries (e.g., `search_places("halal seafood restaurant", near=Hotel, party=6)`).
2. **Multi-Source Retrieval:** Queries relevant APIs (Google Places for POIs/dining, Booking.com for hotels, Klook for activities).
3. **Hard Filtering:** Eliminates candidates exceeding group budget cap, violating dietary rules, lacking capacity, or closed during visit slots.
4. **Heuristic Scoring:** Ranks viable options via a weighted utility score:

$$\text{Score}(c) = w_1 \cdot \text{Rating} + w_2 \cdot \text{PriceFit} + w_3 \cdot \text{Proximity} + w_4 \cdot \text{PreferenceMatch}$$

   where weights w_i adapt to chat cues (e.g., higher w_2 for budget constraints; negative penalties for previously rejected attributes).
5. **Selection & Standbys:** The top candidate forms the proposed leg, while 2–3 next-best options are retained as instant voting alternatives.

3.5. Destination Discovery: Unset Destinations

When the destination is unspecified (e.g., *"warm place near Hanoi, 3 days, hike + beach, $150/pax"*):

- **Stage A (Candidate Generation):** LLM analyzes constraints (duration $\leq$ 3 days $\rightarrow$ travel $\leq$ 2h; budget \$150 $\rightarrow$ domestic; hike + beach $\rightarrow$ Cat Ba / Ha Long shortlist).
- **Stage B (Live Validation):** Parallel tool checks verify weather (`get_weather`), flight/travel costs (`search_flights`), and attraction accessibility. Surviving top 2–3 destinations are presented to the group for a preliminary consensus vote before detailed itinerary planning.

3.6. Place Selection Reasoning: Spatial & Temporal Coherence

Individual scores do not guarantee a smooth schedule. The agent selects places based on geographic feasibility, realistic travel times, and logical flow:

- **Geographic Clustering:** Groups candidate POIs into spatial clusters relative to an anchor (e.g., hotel). The agent favors cluster-coherent choices (e.g., picking a restaurant 400 m from the morning attraction over a slightly higher-rated one 14 km away to avoid wasted transit).
- **Half-Day Time Blocks:** Partitions days into Morning, Afternoon, and Evening blocks. Stops within a block are restricted to the same geographic cluster.
- **Anti-Zigzag Optimization:** Analyzes route geometry and applies nearest-neighbor sequencing to flexible stops, preventing backtracking across the city.

3.7. Spatial Planning: Routing, Distance, Time & Transport

Spatial planning validates that the itinerary is physically feasible within group time budgets:

1. **Realistic Transit & Timing:** Computes real road distance and travel time via `get_route` with departure timestamps. Peak-hour transitions (7–9 am, 5–7 pm) automatically include a 30–50% traffic buffer.
2. **Feasibility Checks:** Enforces Arrival + Duration $\leq$ Closing Time with $\geq$ 15 min connection slack. Preserves fixed meal slots (lunch 11:30–13:30, dinner 18:00–20:30) and anchors timed-entry attractions.
3. **Transport Mode Selection:** Chooses optimal transit based on group size, distance, and context:

Distance / Context	Default Mode	Rationale
< 1.5 km	Walking	Zero cost, walkable in dense areas, no waiting time.
2–15 km (flexible / youth)	Motorbike / Scooter	Highly agile in traffic, easy parking, cost-effective, scenic exploration.
2–10 km (groups / rain)	Grab / Taxi	Weather-safe, convenient point-to-point transit for small groups.
10–40 km (≥ 5 pax)	Private Van / Car	Keeps group together, lower cost than multi-taxi convoy.
> 40 km	Bus / Flight	Optimizes transit duration versus budget limits.

Override: Modes dynamically adjust if the group specifies a preference (e.g., *"rent motorbikes"*) or during adverse weather.

Example Spatial Optimization (Da Nang, 6 pax): Rather than scheduling distant attractions on the same morning, the agent anchors Bà Nà Hills (35 km) for the morning with on-site lunch (saving a 70 km round-trip), returns off-peak at 14:00 by private van, and assigns the nearby beach for sunset. Total daily transit is reduced from 180 min to 90 min.

3.8. Agentic Reasoning Loop

TripMate AI runs an adaptive **Think** $\rightarrow$ **Act** $\rightarrow$ **Observe** loop rather than a rigid pipeline:

1. **Think:** Evaluates the structured session state (Section 3.9) to choose the single next best action (call a tool, query the group, or emit proposal) using DeepSeek-Pro.
2. **Act:** Executes the selected action (e.g., invoke `search_hotels` or Clarification skill).
3. **Observe:** Parses output, updates state, and evaluates stop conditions.
4. **Stop Condition:** Terminates when the itinerary meets all constraints, consensus is reached, or round limits (Section 3.10) are met.

The 5 UX stages in Figure 1 provide user-facing checkpoints, while internal execution remains dynamic and dependency-driven.

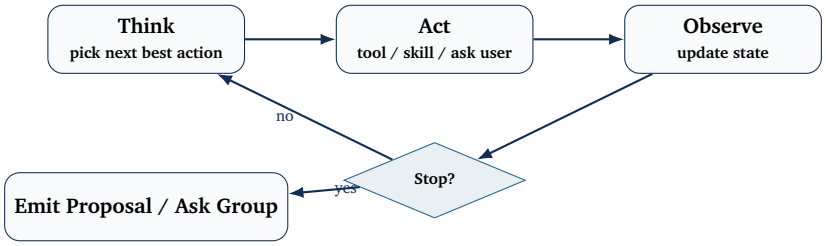

Figure 2: Internal Think-Act-Observe loop: the agent dynamically determines actions each turn until constraints and consensus are satisfied (Section 3.10)

3.9. Context Management

- Maintains a running **structured JSON state** (extracted constraints, budget, booked legs) updated each turn, avoiding bloated token re-transmission.
- The itinerary is maintained as a stateful object of uniquely ID'd legs, enabling atomic local repairs.

3.10. Retry & Error Handling

- External API calls allow up to **2 retries** with exponential backoff before falling back to cached data (Redis, TTL 6–24h) with an "outdated" indicator.
- Unresolvable tool failures are stated transparently to avoid hallucinations.
- Clarification interactions are capped at **3 rounds**; unaddressed inputs default to sensible heuristics.

3.11. Learning Feedback Loop

- Rejection reasons (price, distance, style) are logged in the session state.
- Subsequent candidate proposals filter out attributes matching rejected patterns.
- Feedback is strictly session-scoped, resetting for subsequent trips.

3.12. Worked Examples

Scenario A — Known Destination (Da Nang, 3 days, 6 pax):

- *Turn 1 (Input):* Dates/destination known; diet (halal) parsed from chat → Agent asks only for missing budget.
- *Turn 2 (Research):* Parallel lookup (`search_hotels`, `search_places`, `get_weather`) → Filters budget/capacity, selects top hotel + 2 backups.
- *Turn 3 (Spatial Planning):* Clusters POIs → Anchors Bà Nà Hills (35 km) to Day 2 morning, separates Marble Mountains to Day 3 to prevent zigzagging.
- *Turn 4 (Routing):* `get_route` assigns Grab van (6 pax), applies peak-hour buffers → Validates opening hours → Generates proposal cards.
- *Turn 5 (Repair):* Member downvotes lunch spot → Local-repair immediately injects pre-scored halal alternative.

Scenario B — Unset Destination ("Warm, near Hanoi, 3 days, hike + beach"):

- *Turn 1 (Discovery):* LLM generates candidate shortlist (Cat Ba, Ha Long, Ninh Binh) balancing hike/beach criteria.
- *Turn 2 (Validation):* Checks weather and travel costs in parallel → Drops high-rain candidate → Presents top 2 to group vote.
- *Turn 3 onwards:* Group selects Cat Ba → Resumes standard itinerary research and spatial planning.

4. Backend Architecture

- **Runtime:** Python 3.11+, FastAPI (async, auto OpenAPI docs).
- **Database:** PostgreSQL + SQLAlchemy 2.0 (async) + Alembic migrations.

- **Cache/Broker:** Redis — caches tool results, rate-limits, WebSocket pub/sub.
- **Background jobs:** Celery / BackgroundTasks for PDF export (WeasyPrint) and email.
- **Settlement engine:** 3 tables (`expenses`, `expense_splits`, `settlements`); greedy matching on net balances $\text{Net}_u = \sum \text{Paid}_u - \sum \text{Owed}_u$ minimizes transfer count.

5. Frontend

React 18 + Vite + TypeScript, Tailwind CSS + shadcn/ui, Zustand for state, TanStack Query for caching/optimistic updates. Responsive across mobile, tablet, desktop.

6. Deployment & Security

- **Pipeline:** Docker Compose (local/staging), Vercel (frontend CDN), Render/VPS (backend), GitHub Actions (CI).
- **Security:** bcrypt password hashing, JWT access + Redis-rotated refresh tokens, RBAC, `bleach` sanitization, Pydantic v2 schema validation on all AI output.
- **Data privacy:** all trip-specific information (dietary needs, budget, passport details) lives only in the session chat and the agent's session state; it is not stored in permanent user profiles. Access is scoped by RBAC and not shared outside the room.
- **Rate limiting & audit:** Redis-backed limits on external and AI endpoints; all tool calls and token usage logged in `agent_logs`.

7. Expected Impact

- **Speed:** itinerary consensus in under 15 minutes instead of days of back-and-forth.
- **Fairness:** mathematically resolved schedule/preference conflicts, no single organizer carrying the load.
- **Reliability:** explicit retry, fallback and validation rules keep the agent from silently failing or hallucinating.

TripMate AI turns the chaos of group planning into a seamless, collaborative, and intelligent experience.